마중 DB 현황

2026-10-30 · DB 담당 정리

요약

DB 설계와 구조는 완료됐습니다. 기획서의 시나리오 A·B·C를 처음부터 끝까지 표현할 수 있고, 잘못된 데이터는 DB가 직접 막습니다. 남은 일은 **실제 시설 데이터 입력**과 **서버 연결**입니다.

완료된 것

구조

테이블 13개, 뷰 1개(**v_visit_card**), 트리거 3개입니다.

묶음	테이블	역할
시설	facility, access_point, wait_spot, connection	차 대는 곳, 기다리는 곳, 그 사이 걷는 길
방문	trip, visit, participant, candidate, proposal, stage_event, vehicle_update	외출·방문의 지금 상태와 이력
기록	place_rating, destination_history	장소 평가, 다음 추천용 방문 기록

설계 원칙: 후보 = access_point + wait_spot + connection / 현재 상태는 visit, 이력은 이벤트 테이블 / 제약은 DB에 / version으로 충돌 방지 / 여러 방문은 trip으로 묶기

시나리오 지원

시나리오	방문 구성	확인 결과 (마지막 version)
A. 목적지에서 가족 픽업	pickup 1개, 만난 뒤 같은 시설에 주차	6
B. 함께 출발	dropoff → pickup 2개를 trip으로 묶음, 외출 링크 하나로 계속 봄	6 / 3
C. 픽업 후 다른 장소로	pickup → parking 2개를 trip으로 묶음, 비 오면 실내 주차 추천	5 / 3

DB가 막는 실수

실수	막는 장치
없는 mode, 상태, purpose 값	CHECK
같은 요청을 두 번 보냄	request_id UNIQUE
한 방문에 대기 중 제안 2개	ux_proposal_pending
같은 용도에 1순위 후보 2개	ux_candidate_rank
추천된 만남 후보에 대기 장소가 없음	CHECK
주차 후보를 약속 장소로, 다른 방문의 후보를 저장	trg_visit_candidate_insert / trg_visit_candidate_update
없는 시설, 외출, 후보를 가리킴	외래키
두 사람이 동시에 수정	version (서버가 WHERE version = ? 로 사용)

파일

파일	내용
majung_db/01_schema.sql	테이블, 인덱스, 트리거, 뷰
majung_db/02_seed_facility.sql	시설 1 (예시 쇼핑몰)
majung_db/03_demo_visit.sql	테스트 방문 1건 (비 오는 날 픽업)
majung_db/04_queries.sql	화면별 핵심 쿼리 Q1~Q11 (서버 담당 참고용)
majung_db/05_seed_scenarios.sql	시설 2 (예시 미술관), 시나리오 C용
majung_db/06_scenarios_abc.sql	시나리오 A·B·C 단계별 실행
majung_db/README.md	실행 순서, 설계 요점
majung_db_guide.pdf	열 설명과 시나리오 따라하기 (입문용)

남은 일

1. 실제 시설 데이터 (필수) · 시설 데이터 담당

지금 시설 2곳은 이름과 좌표가 임시값입니다. 실제 서비스할 시설마다 아래를 조사해서 넣어야 합니다.

- 승하차 구역과 주차장: 좌표, 지붕, 승하차 허용, 정차 제한, 진입 방법
- 대기 장소: 층, 실내 여부, 의자
- 동선: 걷는 시간, 유모차 가능, 계단 유무, 비를 피할 수 있는지

02_seed_facility.sql과 같은 형식으로 쓰면 됩니다.

2. 서버 연결 규칙 (필수) · 서버 담당

DB가 잘못된 데이터를 막지만, 서버가 아래 규칙대로 써야 에러 없이 동작합니다.

1. 연결 직후 **PRAGMA foreign_keys = ON** 을 실행합니다. SQLite는 기본값이 꺼져 있습니다.
2. visit을 고칠 때는 항상 **WHERE version = ?** 를 붙이고 version을 1 올립니다. 바뀐 줄이 0이면 409로 응답합니다.
3. 방문은 confirmed_candidate_id와 parking_candidate_id를 비워서 만들고, 후보를 저장한 뒤 UPDATE로 채웁니다.
4. 상태 순서(created → moving → ...) 검사는 서버가 맡습니다. 시나리오마다 흐름이 달라 DB에 고정하지 않았습니다.
5. 같은 동작은 같은 request_id로 보냅니다. UNIQUE 에러가 나면 이미 처리된 요청이므로 현재 상태를 그대로 응답합니다.

```
const db = new Database('majung.db');
db.pragma('foreign_keys = ON'); // 연결 직후 필수
```

3. 앱·서버 로직 (DB 범위 밖)

추천 점수 계산, 날씨 판단, ETA 계산은 앱과 서버의 일입니다. DB는 결과(score, reasons_json, weather_json, last_eta_sec)를 저장만 합니다.

4. 나중에 정할 것 (P1 이후)

- 만료된 방문(expires_at 지남)을 지울지 남길지
- 외출 링크(trip.share_token)를 실제로 쓸지, 방문 링크만 쓸지

- PostgreSQL로 옮길 때 트리거 문법을 다시 쓰기 (INTEGER 0/1 → BOOLEAN, TEXT 시각 → TIMESTAMPTZ도 함께)

다시 확인하는 법

스키마를 고친 뒤에는 DB를 새로 만들고 시나리오를 돌려서 확인합니다. 저장소 폴더(majung-db)에서 PowerShell로 실행합니다. 두 번째 줄은 winget으로 sqlite3를 설치한 경우에만 필요합니다.

```
cd majung-db
$env:Path += ";$env:LOCALAPPDATA₩Microsoft₩WinGet₩Packages₩SQLite.SQLite_Microsoft.Winget.Source_8wekyb3d8bbwe"
Remove-Item check.db* -ErrorAction SilentlyContinue
sqlite3 check.db ".read majung_db/01_schema.sql" ".read majung_db/02_seed_facility.sql"
sqlite3 check.db ".read majung_db/05_seed_scenarios.sql" ".read majung_db/06_scenarios_abc.sql"
```

마지막 표에서 version이 A 6, B 6·3, C 5·3이면 모든 단계가 성공한 것입니다. 하나라도 낮으면 그 시나리오의 version 조건이 어긋난 것입니다.